



SHARDA
UNIVERSITY
Beyond Boundaries

PROJECT SYNOPSIS

Summer Training - 2026

BACHELOR OF TECHNOLOGY

Computer Science & Engineering

SUBMITTED BY

Name - Prince Kumar

System ID - 2023284684

SUPERVISOR

Mr. Aditya Mishra

**DEPARTMENT OF COMPUTER SCIENCE &
ENGINEERING,
SHARDA SCHOOL OF ENGINEERING AND
TECHNOLOGY,
SHARDA UNIVERSITY, GREATER NOIDA**

Problem Statement 2: Design Parking Lot

Objective

Design software for a multi-floor parking lot capable of handling different vehicle types.

Functional Requirements

2.1 Supported Vehicle Types

- Motorcycle — assigned to compact/two-wheeler spots
- Car — assigned to standard/compact spots
- Truck — assigned to large vehicle spots
- Electric Vehicles (EV) — assigned to EV-enabled charging spots (any vehicle category)

2.2 Core Features

- Entry Gate — vehicle identification (ANPR/manual), spot lookup, ticket issuance
- Exit Gate — ticket validation, fee computation, payment collection, gate release
- Assign Parking Spot — real-time allocation based on vehicle type and chosen strategy
- Release Parking Spot — frees the spot immediately on vehicle exit/payment completion
- Generate Ticket — unique ticket per entry with vehicle, spot, and timestamp metadata
- Calculate Parking Fee — pluggable pricing rules (flat / hourly / vehicle-type based)
- Payment — supports cash, card, UPI/wallet, and pre-paid passes via a Payment Gateway
- Display Available Slots — real-time floor-wise and lot-wise availability boards and APIs
- Admin Dashboard — occupancy, revenue, gate health, manual override and reporting

Non Functional Requirements

- High Availability — target 99.95%+ uptime via multi-AZ deployment and service redundancy
- Fast Spot Allocation — sub-100ms allocation decisions using an in-memory cache
- No Duplicate Allocation — atomic, lock-based or compare-and-swap allocation guarantees
- Scalable to Multiple Cities — multi-tenant, horizontally scalable, partitioned by city/lot ID

BOE Tasks

4.1 Traffic Estimate

Assumptions: a mid-size multi-city operator with 500 parking lots, average 800 vehicles/day per lot.

Metric	Assumption / Formula	Estimate
Lots	Across multiple cities	500 lots
Vehicles/day (per lot)	Avg occupancy turnover	~800 vehicles/day
Vehicles/day (system-wide)	500×800	~400,000 vehicles/day
Peak vehicles/hour (per lot)	~15% of daily volume in peak 2 hrs	~60–80 vehicles/hour
Peak vehicles/hour (system-wide)	$500 \text{ lots} \times 70 \text{ (avg)}$	~35,000 vehicles/hour
Entries/sec (system, peak)	$35,000 / 3600$ (entries $\approx$ 50% of events)	~5 entries/sec
Exits/sec (system, peak)	Symmetric to entries	~5 exits/sec
Ticket Generation/sec	1 per entry	~5 tickets/sec
Payments/sec	1 per exit	~5 payments/sec
Available Slot Queries/sec	Display boards + mobile apps, ~20x read amplification	~100 queries/sec

Table 1: Traffic & throughput estimate (steady multi-city peak)

Design implication: write QPS (entries/exits/tickets/payments) is low (~15–20 ops/sec system-wide at peak) but read QPS for availability queries is an order of magnitude higher, justifying a Redis-backed read cache in front of the SQL database.

4.2 Network Bandwidth Estimate

Request Type	Avg Payload Size	Rate (peak)	Bandwidth
Ticket Request/Response	~1 KB (JSON: vehicle, spot, timestamps)	5/sec	~5 KB/sec
Payment Request/Response	~1.5 KB (amount, method, status)	5/sec	~7.5 KB/sec
Slot Display Updates	~2 KB (per floor availability snapshot)	100/sec	~200 KB/sec
Total (approx., with overhead)	—	—	~250–300 KB/sec system-wide

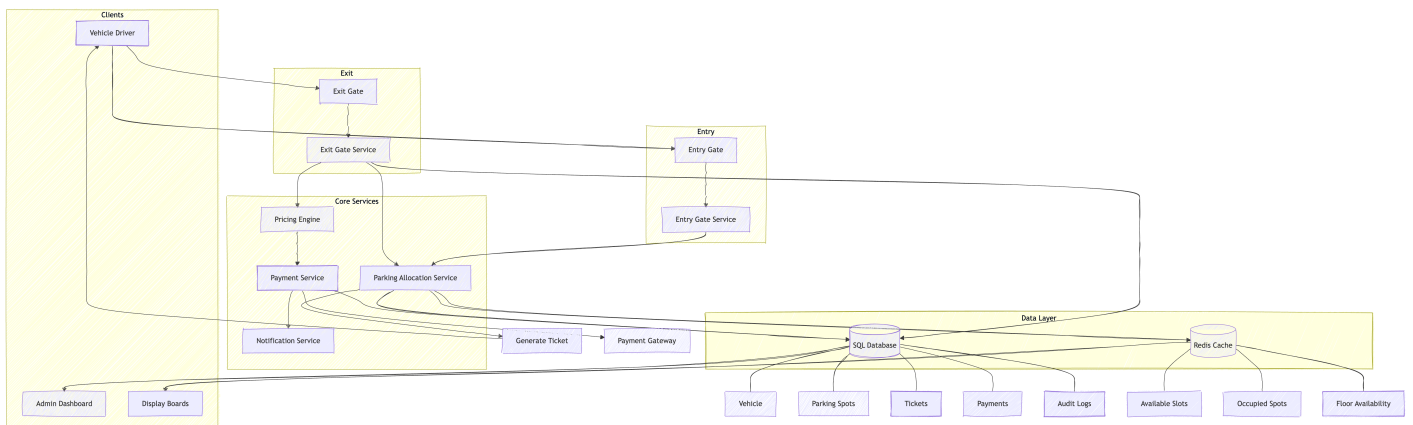
Table 2: Network bandwidth estimate at peak load

4.3 Storage Estimate

Entity	Record Size	Volume	Yearly Storage
Vehicles	~0.5 KB	~5M unique vehicles (cumulative)	~2.5 GB
Tickets	~1 KB	$400\text{K/day} \times 365$	~146 GB/year
Payments	~1 KB	$400\text{K/day} \times 365$	~146 GB/year
Logs (access/audit/system)	~0.5 KB per event, 5x events per transaction	$2\text{M events/day} \times 365$	~365 GB/year
Total (with indexes, ~1.3x overhead)	—	—	~850 GB/year

Table 3: Storage estimate (system-wide, per year)

Recommendation: hot data (active tickets, last 90 days of payments) stays in the primary SQL cluster; older tickets/payments/logs are archived to cold object storage (e.g., S3/Glacier-class tiers) with a time-based partitioning strategy on ticket/payment tables.



5. High-Level Design (HLD)

6. Low-Level Design (LLD)

